CNNP Skill 2: Three-Point Capture

Name: Tejaswin Amara Roll No.: 2520090104

1. Install tools

```
sudo apt update
sudo apt install -y tshark iperf3 mtr-tiny iproute2 wireshark netcat-openbsd
```

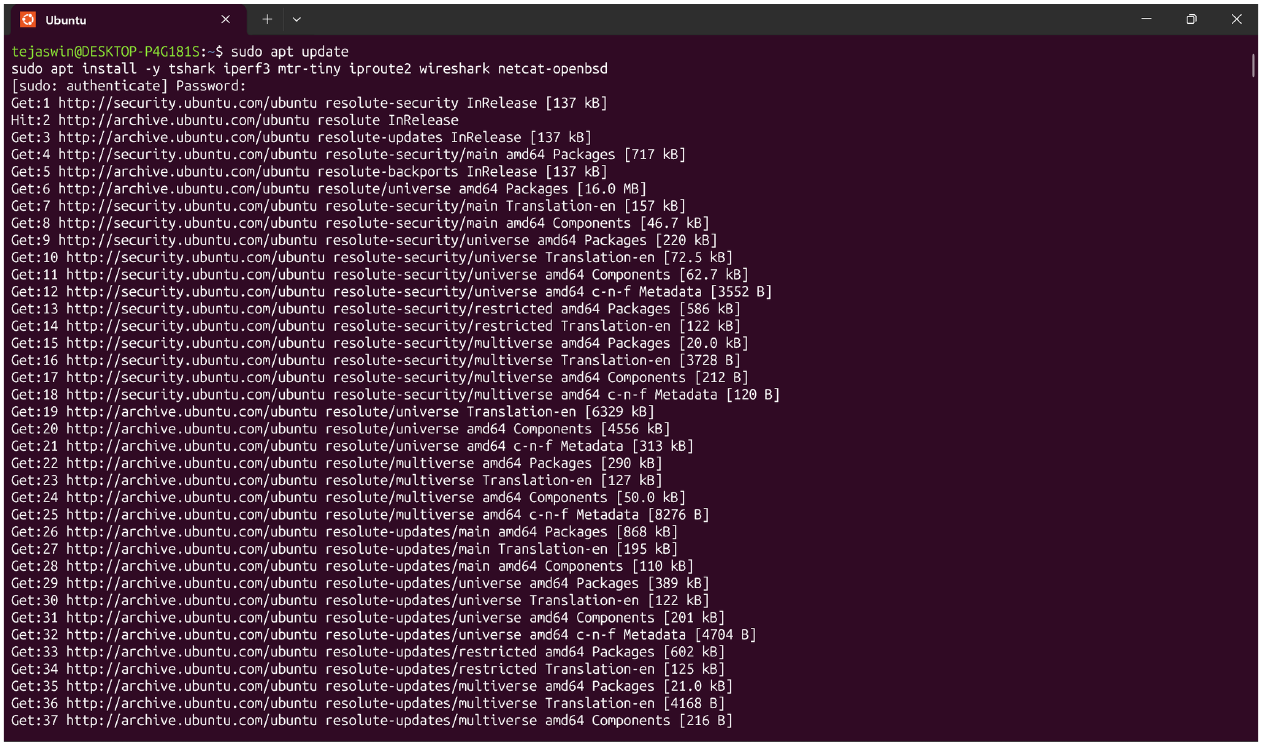


Figure: apt update and install output

2. Build namespaces and veth links

```
sudo ip netns add host1
sudo ip netns add router
sudo ip netns add host2
sudo ip netns list
sudo ip link add veth-h1 type veth peer name veth-r1
sudo ip link add veth-r2 type veth peer name veth-h2
sudo ip link set veth-h1 netns host1
sudo ip link set veth-r1 netns router
sudo ip link set veth-r2 netns router
sudo ip link set veth-h2 netns host2
```

Observation: host2, router and host1 are listed as namespaces. Four veth endpoints are created and moved into their namespaces.

```
tejaswin@DESKTOP-P4G181S:~$ sudo ip netns add host1
sudo ip netns add router
sudo ip netns add host2
sudo ip netns list
host2
router
host1
```

Figure: Namespace creation and list

```
tejaswin@DESKTOP-P4G181S:~$ sudo ip link add veth-h1 type veth peer name veth-r1
sudo ip link add veth-r2 type veth peer name veth-h2

sudo ip link set veth-h1 netns host1
sudo ip link set veth-r1 netns router
sudo ip link set veth-r2 netns router
sudo ip link set veth-h2 netns host2
```

Figure: veth pair creation and assignment

3. IP addressing and routing

```
sudo ip netns exec host1 ip addr add 10.0.1.1/30 dev veth-h1
sudo ip netns exec host1 ip link set veth-h1 up
sudo ip netns exec router ip addr add 10.0.1.2/30 dev veth-r1
sudo ip netns exec router ip addr add 10.0.2.1/30 dev veth-r2
sudo ip netns exec host2 ip addr add 10.0.2.2/30 dev veth-h2
sudo ip netns exec router sysctl -w net.ipv4.ip_forward=1
sudo ip netns exec host1 ip route add 10.0.2.0/30 via 10.0.1.2
sudo ip netns exec host2 ip route add 10.0.1.0/30 via 10.0.2.1
```

Observation: IP forwarding is enabled (net.ipv4.ip_forward = 1). Static routes are set on host1 and host2.

```
tejaswin@DESKTOP-P4G181S:~$ # Configure host1
sudo ip netns exec host1 ip addr add 10.0.1.1/30 dev veth-h1
sudo ip netns exec host1 ip link set veth-h1 up
sudo ip netns exec host1 ip link set lo up

# Configure router
sudo ip netns exec router ip addr add 10.0.1.2/30 dev veth-r1
sudo ip netns exec router ip addr add 10.0.2.1/30 dev veth-r2
sudo ip netns exec router ip link set veth-r1 up
sudo ip netns exec router ip link set veth-r2 up
sudo ip netns exec router ip link set lo up

# Configure host2
sudo ip netns exec host2 ip addr add 10.0.2.2/30 dev veth-h2
sudo ip netns exec host2 ip link set veth-h2 up
sudo ip netns exec host2 ip link set lo up
```

Figure: Host and router address configuration

```
tejaswin@DESKTOP-P4G181S:~$ sudo ip netns exec router sysctl -w net.ipv4.ip
_forward=1
sudo ip netns exec host1 ip route add 10.0.2.0/30 via 10.0.1.2
sudo ip netns exec host2 ip route add 10.0.1.0/30 via 10.0.2.1
net.ipv4.ip_forward = 1
```

Figure: IP forwarding and static route setup

4. Baseline ping (host1 to host2)

```
sudo ip netns exec host1 ping -c 3 10.0.2.2
```

Measurement	Value
TTL on replies	63 (one router hop from 64)
RTT min / avg / max	0.043 / 0.048 / 0.052 ms
Packet loss	0%

```
tejaswin@DESKTOP-P4G181S:~$ sudo ip netns exec host1 ping -c 3 10.0.2.2
PING 10.0.2.2 (10.0.2.2) 56(84) bytes of data.
64 bytes from 10.0.2.2: icmp_seq=1 ttl=63 time=0.050 ms
64 bytes from 10.0.2.2: icmp_seq=2 ttl=63 time=0.043 ms
64 bytes from 10.0.2.2: icmp_seq=3 ttl=63 time=0.052 ms

--- 10.0.2.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2051ms
rtt min/avg/max/mdev = 0.043/0.048/0.052/0.003 ms
```

Figure: Ping output from host1 to host2

5. Netem delay and loss on veth-r2

```
sudo ip netns exec router tc qdisc add dev veth-r2 root netem delay 20ms loss 1%
sudo ip netns exec router tc qdisc show dev veth-r2
```

Observation: qdisc netem 8001 with delay 20ms and loss 1%.

```
tejaswin@DESKTOP-P4G181S:~$ sudo ip netns exec router tc qdisc add dev
veth-r2 root netem delay 20ms loss 1%
sudo ip netns exec router tc qdisc show dev veth-r2
qdisc netem 8001: root refcnt 9 limit 1000 delay 20ms loss 1% seed 1619
7084392192552354
```

Figure: netem configuration on veth-r2

6. Three-point capture

```
sudo ip netns exec host1 tshark -i veth-h1 -w /tmp/cap1.pcap
sudo ip netns exec router tshark -i veth-r1 -w /tmp/cap2.pcap
sudo ip netns exec host2 tshark -i veth-h2 -w /tmp/cap3.pcap
```

Observation: capture counters show 13 packets on veth-h1, 13 on veth-r1 and 14 on veth-h2 at the time of each screenshot.

```
tejaswin@DESKTOP-P4G181S:~$ sudo ip netns exec host1 tshark -i veth-h1 -w /tmp/cap1.pcap
[sudo: authenticate] Password:
Running as user "root" and group "root". This could be dangerous.
Capturing on 'veth-h1'
13
```

Figure: Capture started on veth-h1 (host1)

```
tejaswin@DESKTOP-P4G181S:~$ sudo ip netns exec router tshark -i veth-r1 -w /tmp/cap2.pcap
[sudo: authenticate] Password:
Running as user "root" and group "root". This could be dangerous.
Capturing on 'veth-r1'
13
```

Figure: Capture started on veth-r1 (router)

```
tejaswin@DESKTOP-P4G181S:~$ sudo ip netns exec host2 tshark -i veth-h2 -w /tmp/cap3.pcap
[sudo: authenticate] Password:
Running as user "root" and group "root". This could be dangerous.
Capturing on 'veth-h2'
14
```

Figure: Capture started on veth-h2 (host2)

7. Netcat transfer on port 1234

```
sudo ip netns exec host2 nc -l 1234
sudo ip netns exec host1 nc 10.0.2.2 1234
```

Observation: The Netcat message was sent from host1 to host2 on TCP port 1234.

```
tejaswin@DESKTOP-P4G181S:~$ sudo ip netns exec host2 nc -l 1234
[sudo: authenticate] Password:
sudo chmod 644 /tmp/cap1.pcap /tmp/cap2.pcap /tmp/cap3.pcap
ls -lh /tmp/cap1.pcap /tmp/cap2.pcap /tmp/cap3.pcap
```

Figure: Netcat listener on host2

```
tejaswin@DESKTOP-P4G181S:~$ sudo ip netns exec host1 nc 10.0.2.2 1234
[sudo: authenticate] Password:
sudo chmod 644 /tmp/cap1.pcap /tmp/cap2.pcap /tmp/cap3.pcap
ls -lh /tmp/cap1.pcap /tmp/cap2.pcap /tmp/cap3.pcap
```

Figure: Netcat sender on host1

8. Capture file sizes

```
sudo chmod 644 /tmp/cap1.pcap /tmp/cap2.pcap /tmp/cap3.pcap
ls -lh /tmp/cap1.pcap /tmp/cap2.pcap /tmp/cap3.pcap
```

Observation: cap1, cap2 and cap3 are each 1.7K.

```
tejaswin@DESKTOP-P4G181S:~$ sudo chmod 644 /tmp/cap1.pcap /tmp/cap2.pcap /tmp/cap3.pcap
ls -lh /tmp/cap1.pcap /tmp/cap2.pcap /tmp/cap3.pcap
[sudo: authenticate] Password:
-rw-r--r-- 1 root root 1.7K Nov 12 08:03 /tmp/cap1.pcap
-rw-r--r-- 1 root root 1.7K Nov 12 08:03 /tmp/cap2.pcap
-rw-r--r-- 1 root root 1.7K Nov 12 08:04 /tmp/cap3.pcap
```

Figure: ls output for the three capture files

9. TTL distribution per capture point

```
tshark -r /tmp/cap1.pcap -T fields -e ip.ttl -Y ip | sort | uniq -c
tshark -r /tmp/cap2.pcap -T fields -e ip.ttl -Y ip | sort | uniq -c
tshark -r /tmp/cap3.pcap -T fields -e ip.ttl -Y ip | sort | uniq -c
```

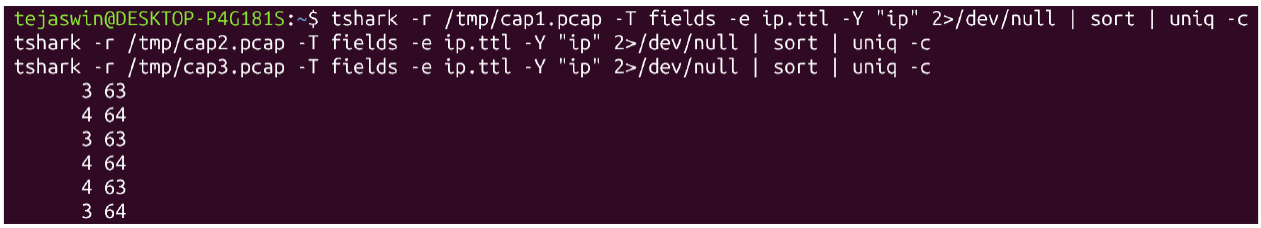


Figure: TTL command output (six lines)

Command order	Printed line	Capture (by command order)
Command 1, line 1	3 63	cap1.pcap
Command 1, line 2	4 64	cap1.pcap
Command 2, line 1	3 63	cap2.pcap
Command 2, line 2	4 64	cap2.pcap
Command 3, line 1	4 63	cap3.pcap
Command 3, line 2	3 64	cap3.pcap

Observation: The printed output has six lines and no file labels, so each pair is assigned by command order. Re-run each command separately to confirm the mapping.

10. Payload and timing

```
tshark -r /tmp/cap2.pcap -Y "tcp.port == 1234" -T fields -e data.text 2>/dev/null
tshark -r /tmp/cap2.pcap -Y "tcp.port == 1234" -T fields -e tcp.payload 2>/dev/null | xxd -r
-p | head -20
tshark -r /tmp/cap1.pcap -Y "tcp.flags.syn==1 && tcp.flags.ack==0" -T fields -e
frame.time_relative
```

Observation: the hex dump of the TCP payload on port 1234 is shown. The SYN relative timestamps are 0.000000000 on each of the three captures.

```
tejaswin@DESKTOP-P4G181S:~$ # View decoded ASCII text
tshark -r /tmp/cap2.pcap -Y "tcp.port == 1234" -T fields -e data.text 2>/dev/null

# View hex and ASCII dump
tshark -r /tmp/cap2.pcap -Y "tcp.port == 1234" -T fields -e tcp.payload 2>/dev/null | xxd | head -20

00000000: 0a0a 0a37 3337 3536 3436 6632 3036 3336  ...7375646f20636
00000010: 3836 6436 6636 3432 3033 3633 3433 3432  86d6f64203634342
00000020: 3032 6637 3436 6437 3032 6636 3336 3137  02f746d702f63617
00000030: 3033 3132 6537 3036 3336 3137 3032 3032  0312e70636170202
00000040: 6637 3436 6437 3032 6636 3336 3137 3033  f746d702f6361703
00000050: 3232 6537 3036 3336 3137 3032 3032 6637  22e70636170202f7
00000060: 3436 6437 3032 6636 3336 3137 3033 3332  46d702f636170332
00000070: 6537 3036 3336 3137 3030 610a 0a36 6337  e706361700a..6c7
00000080: 3332 3032 6436 6336 3832 3032 6637 3436  3202d6c68202f746
00000090: 6437 3032 6636 3336 3137 3033 3132 6537  d702f636170312e7
000000a0: 3036 3336 3137 3032 3032 6637 3436 6437  0636170202f746d7
000000b0: 3032 6636 3336 3137 3033 3232 6537 3036  02f636170322e706
000000c0: 3336 3137 3032 3032 6637 3436 6437 3032  36170202f746d702
000000d0: 6636 3336 3137 3033 3332 6537 3036 3336  f636170332e70636
000000e0: 3137 3030 610a 0a                        1700a..
```

Figure: Decoded payload and hex dump on port 1234

```
tejaswin@DESKTOP-P4G181S:~$ tshark -r /tmp/cap1.pcap -Y "tcp.flags.syn==1 && tcp.flags.ack==0" -T fields -e frame.time_relative 2>/dev/null
tshark -r /tmp/cap2.pcap -Y "tcp.flags.syn==1 && tcp.flags.ack==0" -T fields -e frame.time_relative 2>/dev/null
tshark -r /tmp/cap3.pcap -Y "tcp.flags.syn==1 && tcp.flags.ack==0" -T fields -e frame.time_relative 2>/dev/null
0.000000000
0.000000000
0.000000000
```

Figure: SYN relative timestamps

11. TCP retransmissions

Item	Value
CLI retransmission count (tcp.analysis.retransmission | wc -l)	Not visible in screenshot
Wireshark evidence	TCP Retransmission and TCP Dup ACK packets on port 1234 (SYN, PSH/ACK and ACK retransmissions)

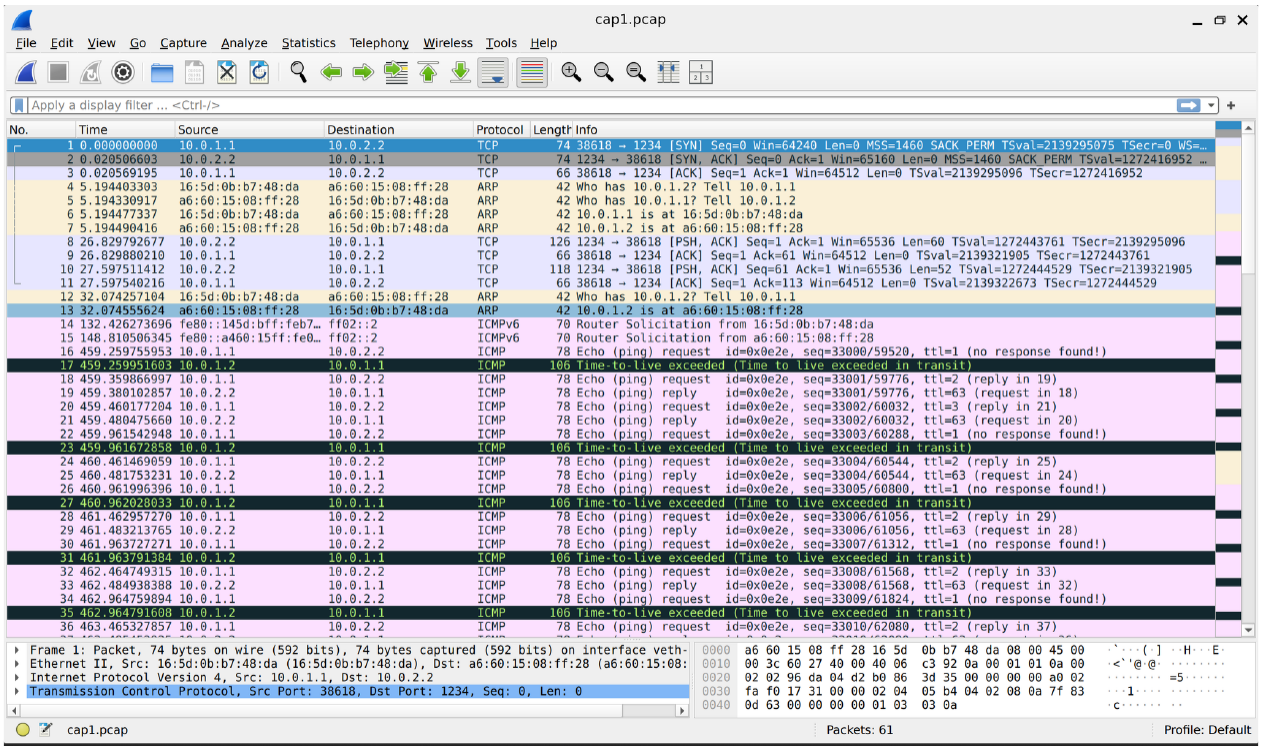


Figure: Wireshark cap1.pcap, unfiltered

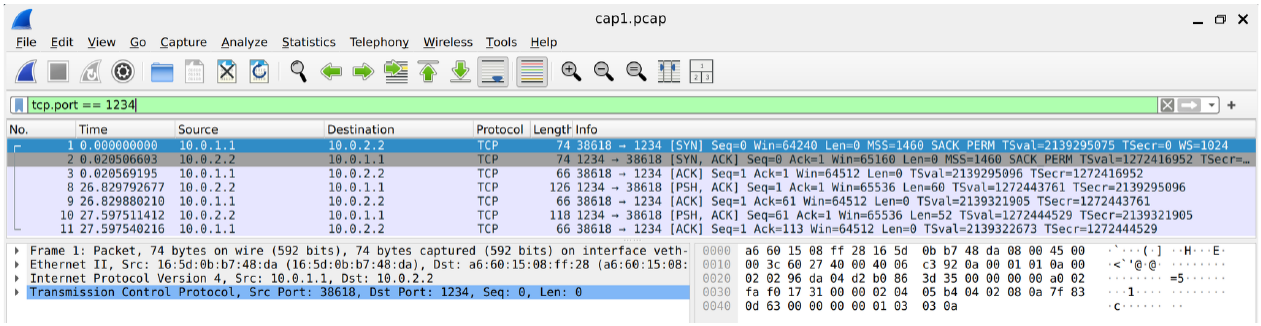


Figure: Wireshark cap1.pcap, filter tcp.port == 1234

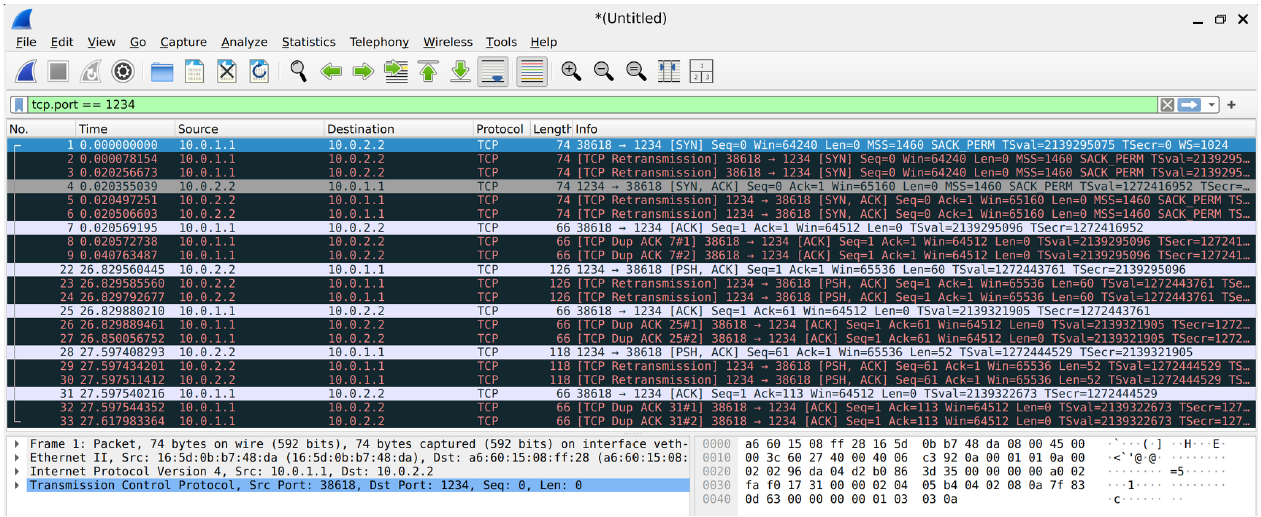


Figure: Wireshark with TCP Retransmission and TCP Dup ACK entries

12. Hop-by-hop RTT (mtr)

```
sudo ip netns exec host1 mtr --report 10.0.2.2
```

Hop	Loss	Avg RTT
10.0.1.2 (router)	0.0%	0.1 ms
10.0.2.2 (host2)	0.0%	20.3 ms

Observation: The 20 ms netem delay on the router egress appears in the host2 hop, consistent with the 20.3 ms average.

```
tejaswin@DESKTOP-P4G181S:~$ sudo ip netns exec host1 mtr --report 10.0.2.2
Start: 2026-11-12T08:09:00+0000
HOST: DESKTOP-P4G181S             Loss%   Snt   Last   Avg  Best  Wrst StDev
  1.|-- 10.0.1.2                   0.0%    10    0.1   0.1   0.1   0.5   0.1
  2.|-- 10.0.2.2                   0.0%    10   20.4  20.3  20.2  20.4   0.1
tejaswin@DESKTOP-P4G181S:~$ wireshark /tmp/cap2.pcap &
[1] 3646
```

Figure: mtr report

13. Cleanup

```
sudo ip netns del host1
sudo ip netns del router
sudo ip netns del host2
sudo ip netns list
```

Observation: the namespaces are deleted and the list is empty.

```
tejaswin@DESKTOP-P4G181S:~$ sudo ip netns del host1
sudo ip netns del router
sudo ip netns del host2
sudo ip netns list
[1]+  Done                    wireshark /tmp/cap2.pcap
```

Figure: Namespace cleanup